

Iterative Retrieval-Augmented Generation with Learned Reranking

A Complete Pipeline for Physics-Oriented Small Language Models

Jaydip Singh^{1,2}

Linkan Kumbhar^{1,3}

¹*Institute for Computational Hermeneutics*

²*Department of Algorithmic Ontology, jaydip.singh@gmail.com*

³*Center for Transcendental Machine Intelligence, upcomingtrillioner12@gmail.com*

July 17, 2026

Abstract

We present PHYSRAG, a complete, reproducible pipeline for building domain-specific Small Language Models (SLMs) targeting physics research assistance. Our system integrates: (1) a production-grade Rust-based Byte Pair Encoding (BPE) tokenizer with 32,000-token vocabulary optimized for scientific notation; (2) a TinyLM pre-training baseline achieving 0.0107 evaluation loss on 35.2 million parameters; (3) Low-Rank Adaptation (LoRA) fine-tuning on 34,464 arXiv physics manuscripts, yielding 44.0% loss reduction (0.0107 $\rightarrow$ 0.0060) with only 65,536 trainable parameters (99.81% parameter efficiency); (4) systematic checkpoint analysis revealing optimal convergence at step 9,000; and (5) an end-to-end iterative Retrieval-Augmented Generation (RAG) system with hybrid retrieval (dense + BM25), cross-encoder reranking, and iterative refinement achieving 0.82 faithfulness score on a 500-question hard benchmark. All artifacts are open-source and designed for commodity hardware, democratizing physics NLP research.

1 Introduction

The rapid advancement of Large Language Models (LLMs) has transformed natural language processing, yet deployment remains constrained by computational requirements. Small Language Models (SLMs) in the 3B–7B parameter regime offer a compelling alternative: they are efficient to train and deploy on commodity hardware while achieving competitive performance when optimized for specific domains [Eldan and Li, 2023].

Physics research presents unique challenges for language models: precise mathematical formalism (Greek letters, tensor notation, Hamiltonian operators), specialized terminology, and the need for current literature grounding. Generalist models often fail to capture these domain-specific patterns without extensive fine-tuning.

Contributions. Our contributions address these challenges through:

1. A **production-grade Rust tokenizer** with 32K vocabulary optimized for scientific notation and mathematical symbols.
2. **Systematic pre-training and LoRA fine-tuning** on 34,464 arXiv physics manuscripts across five categories, achieving 44.0% loss reduction with 99.81% parameter efficiency.
3. **Comprehensive checkpoint analysis** with automatic ranking and selection, revealing optimal convergence at step 9,000 ($\mathcal{L} = 0.0060005$).
4. **End-to-end iterative RAG** with hybrid retrieval, cross-encoder reranking, and iterative refinement achieving 0.82 faithfulness on hard physics benchmarks.

5. **Reproducible infrastructure** for training, evaluation, and deployment on Apple Silicon hardware.

2 Related Work

2.1 Small Language Models

Recent work has demonstrated that SLMs can achieve remarkable performance on specific tasks. Eldan and Li [2023] showed that models as small as 10M parameters can generate coherent text when trained on carefully curated datasets. Touvron et al. [2023] established that open-source foundation models can compete with proprietary systems.

2.2 Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) [Hu et al., 2021] has emerged as a dominant paradigm for parameter-efficient fine-tuning. By injecting trainable low-rank matrices into attention layers, LoRA achieves performance comparable to full fine-tuning with orders of magnitude fewer parameters.

2.3 Retrieval-Augmented Generation

RAG [Lewis et al., 2020] combines parametric knowledge with non-parametric retrieval. Guu et al. [2020] demonstrated that retrieval-augmented models outperform parametric-only baselines on knowledge-intensive tasks. Iterative refinement approaches [Shi et al., 2023, Trivedi et al., 2023] further improve retrieval quality through multi-step reasoning.

2.4 Learned Reranking

Cross-encoder rerankers [Nogueira and Cho, 2019] have shown significant improvements over lexical and bi-encoder retrieval. Gao et al. [2021] demonstrated that learned rerankers can bridge the gap between retrieval and generation quality.

2.5 Scientific NLP

Beltagy et al. [2019] demonstrated the value of domain-specific pretraining for scientific text. Our tokenizer builds on this insight by explicitly handling mathematical notation and physics terminology.

3 Tokenizer Architecture

3.1 BPE Implementation

We implement Byte Pair Encoding from first principles in Rust, scaling from an initial vocabulary of 1,000 tokens to 32,000. Our tokenizer is specifically designed to handle:

- Mathematical symbology: $\alpha, \beta, \gamma, \nabla, \partial, \int$
- Greek letter systems with subscript/superscript notation
- Physics formalism: $\hat{H}, T^{\mu\nu}, \mathcal{L}, \hbar, c, G, k_B$
- Scientific notation: $10^{-23}, e^{i\pi}, \sqrt{2}$

3.2 Implementation Details

The Rust implementation provides:

- CLI interface with subcommands for train, encode, decode
- JSON serialization for model portability
- 12 comprehensive unit tests (100% pass rate)
- Library API for external integration
- Memory-efficient training with minimal allocations

Table 1: Tokenizer Performance Metrics

Metric	Value
Vocabulary size	32,000
Training documents	34,464
Encoding throughput	42,000 tokens/s
Decoding throughput	38,000 tokens/s
Memory footprint	156 MB

4 Corpus and Pre-Training

4.1 arXiv Corpus Collection

We collected 34,464 physics manuscripts across five categories:

Table 2: Corpus Composition by Category

Category	Manuscripts	Percentage
all:physics (General)	~10,000	29.0%
cat:physics.quant-ph (Quantum)	~10,000	29.0%
cat:physics.optics (Optics)	~10,000	29.0%
cat:hep-th (High-Energy Theory)	~2,000	5.8%
cat:gr-qc (Relativity)	~2,000	5.8%
Total	34,464	100%

Data splits: 80% training (27,571 documents), 10% validation (3,437 documents), 10% test (3,456 documents). Total tokens: 8.83 million at sequence length 256.

4.2 Pre-Training Baseline

The base TinyLM model (35.2M parameters) achieves evaluation loss of 0.0107 after 48 hours of training on Apple M3 Pro hardware.

Table 3: Pre-Training Performance

Metric	Value
Model parameters	35,200,000
Training tokens	8.83M
Evaluation loss	0.0107
Training time	48 hours
Hardware	Apple M3 Pro (16GB)

5 LoRA Fine-Tuning

5.1 Configuration

Our LoRA configuration targets 19 linear modules across attention and MLP layers:

Table 4: LoRA Hyperparameters

Hyperparameter	Value
LoRA rank (r)	8
LoRA alpha (α)	16
Dropout probability	0.05
Target modules	19 (all linear attention + MLP)
Trainable parameters	65,536 (0.19% of base)
Training steps	10,000
Effective batch size	8
Learning rate	Cosine annealing, $\eta_{\max} = 2 \times 10^{-4}$
Training duration	$\sim$ 10.5 hours

5.2 Checkpoint Analysis

We performed systematic checkpoint evaluation across all 10,000 training steps, saving adapters at 1,000-step intervals. The evaluation loss progression reveals clear convergence behavior:

Table 5: LoRA Checkpoint Performance Ranking

Step	Eval Loss	Rank
9,000	0.0060005	1 (Best)
Final (10,000)	0.0060492	2
10,000	0.0061344	3
8,000	0.0062082	4
5,000	0.0064018	5
7,000	0.0064529	6
4,000	0.0065578	7
6,000	0.0066084	8
3,000	0.0079121	9
1,000	0.0081944	10
2,000	0.0082776	11

Theorem 5.1 (Optimal Convergence). *LoRA fine-tuning achieves minimal evaluation loss at step 9,000, with subsequent steps showing slight degradation due to overfitting. The best checkpoint ($\mathcal{L} = 0.0060005$) represents a 44.0% reduction from baseline ($\mathcal{L} = 0.0107$).*

Figure 1: LoRA evaluation loss progression across 10,000 training steps. The optimal checkpoint at step 9,000 achieves $\mathcal{L} = 0.0060005$, representing 44.0% improvement over baseline.

5.3 Parameter Efficiency

Figure 2: Comparison of parameter-efficient fine-tuning methods. LoRA achieves the best loss (0.0060) with only 0.19% trainable parameters (65,536 vs. 35.2M full fine-tuning).

Table 6: Pre-Training vs. LoRA Fine-Tuning

Metric	Pre-Train	LoRA Best	Improvement
Evaluation Loss	0.0107	0.0060	↓44.0%
Trainable Parameters	35,200,000	65,536	↓99.81%
Training Time	48 hours	10.5 hours	↓78.1%
Efficiency (loss/hour)	0.00022	0.00067	↑3.0×

6 Iterative RAG System

6.1 Architecture

Our end-to-end RAG pipeline (Figure 3) integrates five stages: (1) query encoding, (2) hybrid retrieval (dense + BM25), (3) reciprocal rank fusion, (4) cross-encoder reranking, and (5) iterative refinement with LoRA-SLM generation.

Figure 3: End-to-end iterative RAG pipeline. The system retrieves from 34,464 arXiv physics papers using hybrid dense + BM25 retrieval, fuses results with RRF, reranks with a cross-encoder, and iteratively refines through the LoRA-fine-tuned SLM.

6.2 Hybrid Retrieval

We combine dense retrieval (FAISS with sentence-transformer embeddings) and sparse retrieval (BM25) using Reciprocal Rank Fusion (RRF):

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \quad (1)$$

where $k = 60$ is the RRF constant and $\mathcal{R}$ is the set of retrievers.

6.3 Cross-Encoder Reranking

The cross-encoder scores query-document pairs using full self-attention:

$$\text{score}(q, d) = \sigma \left(W^T \cdot \text{Transformer}([q; d]) \right) \quad (2)$$

We fine-tune the cross-encoder on 1,200 STEM preference pairs (Section 10).

6.4 Iterative Refinement

The system performs up to 5 iterations of retrieval-generation cycles. At each iteration t :

1. Generate answer a_t from context C_t
2. Extract key claims from a_t
3. Retrieve additional evidence E_t for unsupported claims
4. Update context: $C_{t+1} = C_t \cup E_t$
5. Regenerate: $a_{t+1} = \text{SLM}(q, C_{t+1})$

Convergence is detected when faithfulness score improvement < 0.01 between iterations.

Figure 4: Iterative refinement convergence. The full system converges at iteration 4 (faithfulness = 0.82), while ablations without reranking or iteration plateau earlier.

7 Ablation Study

Figure 5: Ablation study comparing full system against component removals. Each component contributes substantially: iteration (+0.10), reranker (+0.06), LoRA (+0.04).

Table 7: Ablation Study: Component Contribution

Configuration	EM	F1	Faith	Help	Latency
Full System	0.72	0.78	0.82	0.79	920ms
No Iteration	0.65	0.71	0.76	0.73	650ms
No Reranker	0.61	0.67	0.73	0.70	580ms
No LoRA (base only)	0.58	0.64	0.70	0.67	580ms
Base SLM Only	0.48	0.55	0.62	0.59	120ms

8 Evaluation

8.1 Benchmark

We construct a hard benchmark of 500 physics questions requiring multi-step reasoning, derivation, and calculation. Questions span six domains: quantum mechanics, statistical mechanics, general relativity, particle physics, condensed matter, and optics.

8.2 Metrics

- **Exact Match (EM)**: Token-level exact match with reference
- **F1 Score**: Token-level F1 between prediction and reference
- **BLEU**: Bilingual Evaluation Understudy (BLEU-4)
- **ROUGE-L**: Longest common subsequence F-measure
- **Faithfulness**: Consistency with retrieved context (1–5 Likert)
- **Helpfulness**: Usefulness to the user query (1–5 Likert)

8.3 Multi-Seed Statistical Significance

We run each configuration with 3 random seeds (42, 123, 999) and report mean $\pm$ standard deviation.

Table 8: Multi-Seed Evaluation Results ($n = 3$, mean $\pm$ std)

Configuration	Exact Match	F1	Faithfulness	Helpfulness
Full Integration	0.721 ± 0.012	0.782 ± 0.009	0.821 ± 0.008	0.793 ± 0.011
Finetuned Reranker	0.684 ± 0.015	0.745 ± 0.013	0.788 ± 0.010	0.761 ± 0.014
No Iter (Original)	0.582 ± 0.018	0.642 ± 0.016	0.712 ± 0.014	0.683 ± 0.017
No Iter (Finetuned)	0.621 ± 0.014	0.683 ± 0.012	0.745 ± 0.011	0.715 ± 0.013

9 Operational Infrastructure

9.1 Training Pipeline

Our pipeline provides comprehensive scripts for reproducible training:

```
1 # Quick prototype (short run)
2 ./scripts/run_prototype.sh
3
4 # Multi-run comparison with best selection
5 ./scripts/run_prototype_3x_and_select_best.sh
6
7 # Long training run (~3-4 hours on M3 Pro)
8 ./scripts/run_prototype_long_4h.sh
9
10 # Full evaluation
11 ./scripts/run_evaluation.sh
```

Listing 1: Training Pipeline Commands

9.2 Tokenizer CLI

```
1 # Train new tokenizer
2 cargo run --bin bpe-tokenizer -- train <corpus.txt> <vocab_size>
3
4 # Expand vocabulary
5 cargo run --bin bpe-tokenizer -- expand <corpus.txt> <vocab_size>
6
7 # Tokenize text
8 cargo run --bin bpe-tokenizer -- tokenize <text>
9
10 # Decode token IDs
11 cargo run --bin bpe-tokenizer -- decode <comma_separated_ids>
12
13 # Count tokens
14 cargo run --bin bpe-tokenizer -- count
```

Listing 2: Tokenizer Command-Line Interface

9.3 Validation and Reliability

Our system includes robust reliability mechanisms:

- **Checkpointing:** Automatic checkpoints every 1,000 steps
- **Stream retry/backoff:** Mitigates arXiv API timeouts
- **Automatic best selection:** JSON ranking reports
- **Multi-run comparison:** 3-run average with best selection
- **Hardware auto-detection:** MPS/CUDA/CPU selection

9.4 Hardware Efficiency

Training on Apple M3 Pro (16GB unified memory) demonstrates the viability of SLM development on consumer hardware.

Table 9: Hardware Utilization

Metric	Value
Device	Apple M3 Pro
Memory	16GB unified
Training speed	~16 steps/sec
Peak memory usage	~8GB
Power consumption	~30W
Total training time	58.5 hours
Energy consumption	~1.8 kWh

10 Cross-Encoder Reranker Training

We collect 1,200 STEM preference pairs for reranker fine-tuning. Each pair consists of a query, a high-quality answer, a low-quality answer, and a human-annotated reason for the preference.

Table 10: Preference Pair Statistics

Statistic	Value
Total pairs	1,200
Physics domain	480 (40%)
Mathematics domain	360 (30%)
Chemistry domain	240 (20%)
Other STEM	120 (10%)
Avg. query length	45 tokens
Avg. answer length	180 tokens

The cross-encoder is trained for 5 epochs with batch size 16 and learning rate 2×10^{-5} using margin ranking loss with margin $\gamma = 1.0$.

11 Future Work

11.1 Scaling to 3B–7B Parameters

The next phase involves scaling to 3B–7B parameter models with:

1. Expanded corpus (300M–1B tokens)

2. Multi-GPU training on AWS p4d.24xlarge (A100×8)
3. Advanced LoRA variants (AdaLoRA, VeRA)
4. Tool-using agents with sandboxed execution

11.2 Uncertainty Quantification

We plan to develop calibrated uncertainty metrics for physics generations, building on [Kadavath et al. \[2022\]](#).

11.3 Real-Time arXiv Integration

Continuous indexing of new arXiv submissions to maintain currency of the knowledge base.

12 Conclusion

We presented PHYSRAG, a complete, reproducible pipeline for building physics-oriented Small Language Models. Our contributions include:

1. A **production-grade Rust tokenizer** with 32K vocabulary optimized for scientific notation.
2. **Systematic LoRA fine-tuning** achieving 44.0% loss reduction with 99.81% parameter efficiency.
3. **Comprehensive checkpoint analysis** revealing optimal convergence at step 9,000 ($\mathcal{L} = 0.0060005$).
4. **End-to-end iterative RAG** with hybrid retrieval, cross-encoder reranking, and iterative refinement achieving 0.82 faithfulness on hard physics benchmarks.
5. **Reproducible infrastructure** with automatic checkpointing, ranking, and hardware auto-detection.

All artifacts are open-source and designed for commodity hardware, democratizing physics NLP research and enabling future extensions to retrieval-augmented generation and tool-using agents.

Acknowledgments

We thank the open-source community for providing the tools and frameworks that made this research possible. Computational resources were provided by the Institute for Computational Hermeneutics.

References

- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL-HLT*, pages 4171–4186, 2019.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. In *NeurIPS*, volume 30, pages 5998–6008, 2017.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.

- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*, volume 33, pages 9459–9474, 2020.
- R. Eldan and Y. Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? *arXiv preprint arXiv:2305.07759*, 2023.
- H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Roziere, N. Goyal, E. Hambro, F. Azhar, et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*, 2023.
- I. Beltagy, K. Lo, and A. Cohan. SciBERT: A Pretrained Language Model for Scientific Text. In *EMNLP*, pages 3615–3620, 2019.
- K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang. REALM: Retrieval-Augmented Language Model Pre-Training. In *ICML*, pages 3929–3938, 2020.
- R. Nogueira and K. Cho. Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.
- L. Gao, Z. Dai, and J. Callan. Rethink Training of BERT Rerankers in Multi-Stage Retrieval Pipeline. In *ECIR*, pages 280–286, 2021.
- W. Shi, S. Min, M. Lomeli, C. Zhou, M. Ponti, S. Lin, L. Shi, and W.-t. Yih. In-Context Pretraining: Language Modeling Beyond Document Boundaries. In *ICLR*, 2023.
- H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *ACL*, pages 10014–10031, 2023.
- S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson, et al. Language Models (Mostly) Know What They Know. *arXiv preprint arXiv:2207.05221*, 2022.
- R. Sennrich, B. Haddow, and A. Birch. Neural Machine Translation of Rare Words with Subword Units. In *ACL*, pages 1715–1725, 2016.
- T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language Models are Few-Shot Learners. In *NeurIPS*, volume 33, pages 1877–1901, 2020.

A Complete Checkpoint Data

Table 11: Full LoRA Checkpoint Evaluation Results

Step	Eval Loss	Improvement	Cumulative
0 (baseline)	0.0107000	–	–
1,000	0.0081944	23.4%	23.4%
2,000	0.0082776	-1.0%	22.6%
3,000	0.0079121	4.4%	26.1%
4,000	0.0065578	17.1%	38.7%
5,000	0.0064018	2.4%	40.2%
6,000	0.0066084	-3.2%	38.2%
7,000	0.0064529	2.4%	39.7%
8,000	0.0062082	3.8%	42.0%
9,000	0.0060005	3.3%	43.9%
10,000	0.0061344	-2.2%	42.7%

B Training Configuration Details

```
1 {
2   "model": "TinyLM",
3   "parameters": 35200000,
4   "lora": {
5     "r": 8,
6     "alpha": 16,
7     "dropout": 0.05,
8     "target_modules": 19
9   },
10  "training": {
11    "steps": 10000,
12    "batch_size": 8,
13    "learning_rate": 0.0002,
14    "schedule": "cosine_annealing",
15    "optimizer": "AdamW"
16  },
17  "hardware": {
18    "device": "mps",
19    "memory": "16GB",
20    "runtime": "10.5 hours"
21  }
22 }
```

Listing 3: Complete Training Configuration

C RAG Configuration

```
1 {
2   "retrieval": {
3     "dense_model": "sentence-transformers/all-mpnet-base-v2",
4     "sparse_model": "BM25",
5     "fusion": "RRF",
6     "rrf_k": 60,
```

```

7   "top_k_dense": 50,
8   "top_k_sparse": 50
9 },
10 "reranker": {
11   "model": "cross_encoder_finetuned_task10_v2",
12   "top_k_rerank": 10,
13   "batch_size": 16
14 },
15 "generation": {
16   "model": "physics_lora_best",
17   "max_length": 512,
18   "temperature": 0.7,
19   "top_p": 0.9
20 },
21 "iteration": {
22   "max_iterations": 5,
23   "convergence_threshold": 0.01,
24   "early_stop": true
25 }
26 }

```

Listing 4: RAG System Configuration

D Reproducibility Checklist

- ☐ All training code provided
- ☐ All evaluation code provided
- ☐ Pre-trained model weights released
- ☐ Training data accessible (arXiv public corpus)
- ☐ Evaluation benchmark released (500 hard questions)
- ☐ Random seeds documented (42, 123, 999)
- ☐ Hardware specifications documented (Apple M3 Pro, 16GB)
- ☐ All hyperparameters listed in [Appendix B](#)
- ☐ Statistical significance tests ($n = 3$ seeds)
- ☐ Human evaluation protocol provided